

Task on RDS

1.Create MariaDB DB on EC2.

RDS Task 1 – Create MariaDB Database on EC2

Objective

Launch an EC2 instance, install MariaDB, start the database service, and create a database named `devopsdb`.

Step 1: Launch EC2 Instance

In AWS Console:

EC2 → Instances → Launch instances

Configuration used:

Name : MariaDB-EC2
AMI : Amazon Linux 2023
Instance Type : t3.micro
Key Pair : my_server
Public IP : Enabled
Storage : 8 GiB gp3

Launch the instance and wait until it is in the **Running** state.

Step 2: Connect to EC2

From Git Bash:

```
chmod 400 "my_server.pem"
```

Connect using SSH:

```
ssh -i "my_server.pem" ec2-user@<EC2-PUBLIC-DNS>
```

After successful connection, the prompt looks similar to:

[ec2-user@ip-172-31-21-20 ~]\$

Step 3: Install MariaDB

Install the MariaDB server:

```
sudo dnf install mariadb105-server -y
```

Installation completes with:

Complete!

Step 4: Start MariaDB Service

```
sudo systemctl start mariadb
```

Check the service:

```
sudo systemctl status mariadb
```

Expected result:

Active: active (running)

Step 5: Enable MariaDB at Boot

```
sudo systemctl enable mariadb
```

This configures MariaDB to start automatically after an EC2 reboot.

Step 6: Login to MariaDB

```
sudo mariadb
```

Successful login gives:

```
MariaDB [(none)]>
```

The MariaDB version in our lab was:

```
10.5.29-MariaDB
```

Step 7: Create Database

Inside MariaDB:

```
CREATE DATABASE devopsdb;
```

Verify the database:

SHOW DATABASES;

Output:

```
+-----+
| Database      |
+-----+
| devopsdb      |
| information_schema |
| mysql         |
| performance_schema |
+-----+
```

Result

- ✓ EC2 instance launched
- ✓ MariaDB installed
- ✓ MariaDB service started and enabled
- ✓ Connected to MariaDB
- ✓ **devopsdb** database created successfully

Task 1 completed: Create MariaDB DB on EC2.

The screenshot displays the AWS Management Console interface for the EC2 Instances page. The top navigation bar shows the user is logged in as 'chandrashekar' in the 'United States (N. Virginia)' region. The left sidebar contains the navigation menu with options like Dashboard, AWS Global View, Events, and various EC2 instance management tools. The main content area shows a list of instances under the heading 'Instances (1/3) Info'. The instance 'MariaDB-EC2' is selected, and its details are shown below. The instance is in the 'Running' state, has a public IP address of 100.55.39.32, and a public DNS name of ec2-100-55-39-32.compute-1.amazonaws.com. The instance type is t3.micro, and it is located in the us-east-1c availability zone. The instance summary section provides a quick overview of the instance's configuration, including its ID, IP addresses, hostname type, and VPC ID.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4
MariaDB-EC2	i-0af0ed2e2327db872	Running	t3.micro	Initializing	us-east-1c	ec2-100-55-39-32.com...	100.55.39
LB-Web-Server-1	i-08b18795ea904c4f6	Terminated	t3.micro	-	us-east-1a	-	-
LB-Web-Server-2	i-0e0be2ef08ed10318	Terminated	t3.micro	-	us-east-1a	-	-

i-0af0ed2e2327db872 (MariaDB-EC2)

Instance summary

Instance ID	Public IPv4 address	Private IPv4 addresses
i-0af0ed2e2327db872	100.55.39.32 open address	172.31.21.20

Instance state

Running

Hostname type

IP name: ip-172-31-21-20.ec2.internal

Private IP DNS name (IPv4 only)

ip-172-31-21-20.ec2.internal

Instance type

t3.micro

Auto-assigned IP address

VPC ID

Public DNS

ec2-100-55-39-32.compute-1.amazonaws.com | [open address](#)

Elastic IP addresses

-

AWS Compute Optimizer finding

```

Complete!
[root@ip-172-31-21-20 ec2-user]# sudo systemctl start mariadb
[root@ip-172-31-21-20 ec2-user]# sudo systemctl status mariadb
● mariadb.service - MariaDB 10.5 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; disabled; preset: disabled)
   Active: active (running) since Thu 2026-09-24 11:31:04 UTC; 11s ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 27053 ExecStartPre=/usr/libexec/mariadb-check-socket (code=exited, status=0/SUCCESS)
   Process: 27089 ExecStartPre=/usr/libexec/mariadb-prepare-db-dir mariadb.service (code=exited, status=0/SUCCESS)
   Process: 27230 ExecStartPost=/usr/libexec/mariadb-check-upgrade (code=exited, status=0/SUCCESS)
  Main PID: 27214 (mariabdd)
    Status: "Taking your SQL requests now..."
     Tasks: 15 (limit: 1059)
    Memory: 66.4M
       CPU: 521ms
   CGroup: /system.slice/mariadb.service
           └─27214 /usr/libexec/mariabdd --basedir=/usr

Sep 24 11:31:04 ip-172-31-21-20.ec2.internal mariadb-prepare-db-dir[27170]: The second is mysql@localhost, it has no password either, but
Sep 24 11:31:04 ip-172-31-21-20.ec2.internal mariadb-prepare-db-dir[27170]: you need to be the system 'mysql' user to connect.
Sep 24 11:31:04 ip-172-31-21-20.ec2.internal mariadb-prepare-db-dir[27170]: After connecting you can set the password, if you would need to be
Sep 24 11:31:04 ip-172-31-21-20.ec2.internal mariadb-prepare-db-dir[27170]: able to connect as any of these users with a password and without sudo
Sep 24 11:31:04 ip-172-31-21-20.ec2.internal mariadb-prepare-db-dir[27170]: See the MariaDB Knowledgebase at https://mariadb.com/kb
Sep 24 11:31:04 ip-172-31-21-20.ec2.internal mariadb-prepare-db-dir[27170]: Please report any problems at https://mariadb.org/jira
Sep 24 11:31:04 ip-172-31-21-20.ec2.internal mariadb-prepare-db-dir[27170]: The latest information about MariaDB is available at https://mariadb.org/
Sep 24 11:31:04 ip-172-31-21-20.ec2.internal mariadb-prepare-db-dir[27170]: Consider joining MariaDB's strong and vibrant community:
Sep 24 11:31:04 ip-172-31-21-20.ec2.internal mariadb-prepare-db-dir[27170]: https://mariadb.org/get-involved/
Sep 24 11:31:04 ip-172-31-21-20.ec2.internal systemd[1]: Started mariadb.service - MariaDB 10.5 database server.
[root@ip-172-31-21-20 ec2-user]#

```

root@ip-172-31-21-20/home/ec2-user

```

Complete!
[root@ip-172-31-21-20 ec2-user]# sudo systemctl start mariadb
[root@ip-172-31-21-20 ec2-user]# sudo systemctl status mariadb
● mariadb.service - MariaDB 10.5 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; disabled; preset: disabled)
   Active: active (running) since Thu 2026-09-24 11:31:04 UTC; 11s ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 27053 ExecStartPre=/usr/libexec/mariadb-check-socket (code=exited, status=0/SUCCESS)
   Process: 27089 ExecStartPre=/usr/libexec/mariadb-prepare-db-dir mariadb.service (code=exited, status=0/SUCCESS)
   Process: 27230 ExecStartPost=/usr/libexec/mariadb-check-upgrade (code=exited, status=0/SUCCESS)
  Main PID: 27214 (mariabdd)
    Status: "Taking your SQL requests now..."
     Tasks: 15 (limit: 1059)
    Memory: 66.4M
       CPU: 521ms
   CGroup: /system.slice/mariadb.service
           └─27214 /usr/libexec/mariabdd --basedir=/usr

Sep 24 11:31:04 ip-172-31-21-20.ec2.internal mariadb-prepare-db-dir[27170]: The second is mysql@localhost, it has no password
Sep 24 11:31:04 ip-172-31-21-20.ec2.internal mariadb-prepare-db-dir[27170]: you need to be the system 'mysql' user to connect.
Sep 24 11:31:04 ip-172-31-21-20.ec2.internal mariadb-prepare-db-dir[27170]: After connecting you can set the password, if you v
Sep 24 11:31:04 ip-172-31-21-20.ec2.internal mariadb-prepare-db-dir[27170]: able to connect as any of these users with a passwo
Sep 24 11:31:04 ip-172-31-21-20.ec2.internal mariadb-prepare-db-dir[27170]: See the MariaDB Knowledgebase at https://mariadb.co
Sep 24 11:31:04 ip-172-31-21-20.ec2.internal mariadb-prepare-db-dir[27170]: Please report any problems at https://mariadb.org/
Sep 24 11:31:04 ip-172-31-21-20.ec2.internal mariadb-prepare-db-dir[27170]: The latest information about MariaDB is available
Sep 24 11:31:04 ip-172-31-21-20.ec2.internal mariadb-prepare-db-dir[27170]: Consider joining MariaDB's strong and vibrant commu
Sep 24 11:31:04 ip-172-31-21-20.ec2.internal mariadb-prepare-db-dir[27170]: https://mariadb.org/get-involved/
Sep 24 11:31:04 ip-172-31-21-20.ec2.internal systemd[1]: Started mariadb.service - MariaDB 10.5 database server.
[root@ip-172-31-21-20 ec2-user]# sudo systemctl enable mariadb
Created symlink /etc/systemd/system/mysql.service → /usr/lib/systemd/system/mariadb.service.
Created symlink /etc/systemd/system/mysqld.service → /usr/lib/systemd/system/mariadb.service.
Created symlink /etc/systemd/system/multi-user.target.wants/mariadb.service → /usr/lib/systemd/system/mariadb.service.
[root@ip-172-31-21-20 ec2-user]# sudo mariadb
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE devopsdb;
Query OK, 1 row affected (0.000 sec)

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| devopsdb |
| information_schema |
| mysql |
| performance_schema |
+-----+
4 rows in set (0.001 sec)

MariaDB [(none)]> |

```

2.Insert some dummy data.

RDS Task 2 – Insert Dummy Data into MariaDB

Objective

Create a table inside the `devopsdb` database and insert dummy employee records.

Step 1: Select the Database

After logging into MariaDB, select the database created in Task 1:

```
USE devopsdb;
```

Output:

```
Database changed  
MariaDB [devopsdb]>
```

Step 2: Create Employee Table

Create an `employees` table with four columns:

```
CREATE TABLE employees (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(50),  
  department VARCHAR(50),  
  salary DECIMAL(10,2)  
);
```

Output:

```
Query OK, 0 rows affected
```

Here:

- `id` → Unique employee ID and automatically increments.
- `name` → Employee name.

- **department** → Employee department.
- **salary** → Employee salary.

Step 3: Verify the Table

Run:

```
SHOW TABLES;
```

Output:

```
+-----+
| Tables_in_devopsdb |
+-----+
| employees          |
+-----+
```

This confirms that the **employees** table was created successfully.

Step 4: Insert Dummy Data

Insert five employee records:

```
INSERT INTO employees (name, department, salary) VALUES
('Chandra', 'DevOps', 50000.00),
('Rahul', 'Cloud', 55000.00),
('Priya', 'Database', 60000.00),
('Arjun', 'Linux', 48000.00),
('Sneha', 'AWS', 58000.00);
```

Output:

```
Query OK, 5 rows affected
Records: 5 Duplicates: 0 Warnings: 0
```

Step 5: Verify the Dummy Data

Run:

```
SELECT * FROM employees;
```

Output:

```
+----+-----+-----+-----+
| id | name  | department | salary |
+----+-----+-----+-----+
| 1 | Chandra | DevOps    | 50000.00 |
| 2 | Rahul  | Cloud     | 55000.00 |
| 3 | Priya  | Database  | 60000.00 |
| 4 | Arjun  | Linux     | 48000.00 |
| 5 | Sneha  | AWS       | 58000.00 |
+----+-----+-----+-----+

5 rows in set
```

Result

Task 2 completed successfully: the `employees` table was created inside `devopsdb`, and **5 dummy records** were inserted and verified.

```

root@ip-172-31-21-20/home/ec2-user
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE devopsdb;
Query OK, 1 row affected (0.000 sec)

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| devopsdb |
| information_schema |
| mysql |
| performance_schema |
+-----+
4 rows in set (0.001 sec)

MariaDB [(none)]> USE devopsdb;
Database changed
MariaDB [devopsdb]> CREATE TABLE employees (
  -> id INT PRIMARY KEY AUTO_INCREMENT,
  -> name VARCHAR(50),
  -> department VARCHAR(50),
  -> salary DECIMAL(10,2)
  -> );
Query OK, 0 rows affected (0.005 sec)

MariaDB [devopsdb]> SHOW TABLES;
+-----+
| Tables_in_devopsdb |
+-----+
| employees |
+-----+
1 row in set (0.000 sec)

MariaDB [devopsdb]> INSERT INTO employees (name, department, salary) VALUES
  -> ('Chandra', 'DevOps', 50000.00),
  -> ('Rahul', 'Cloud', 55000.00),
  -> ('Priya', 'Database', 60000.00),
  -> ('Arjun', 'Linux', 48000.00),
  -> ('Sneha', 'AWS', 58000.00);
Query OK, 5 rows affected (0.002 sec)
Records: 5 Duplicates: 0 Warnings: 0

MariaDB [devopsdb]> SELECT * FROM employees;
+----+-----+-----+-----+
| id | name  | department | salary |
+----+-----+-----+-----+
| 1  | Chandra | DevOps    | 50000.00 |
| 2  | Rahul  | Cloud     | 55000.00 |
| 3  | Priya  | Database  | 60000.00 |
| 4  | Arjun  | Linux     | 48000.00 |
| 5  | Sneha  | AWS       | 58000.00 |
+----+-----+-----+-----+
5 rows in set (0.000 sec)

MariaDB [devopsdb]>

```

3. Take the backup of dummy data on EC2.

Task 3 – Take Backup of Dummy Data on EC2

For your documentation, use these steps:

1. Exit from MariaDB:

exit;

2. Take a backup of the **devopsdb** database:

```
mariadb-dump -u root devopsdb > devopsdb_backup.sql
```

3. Verify that the backup file was created:

```
ls -lh devopsdb_backup.sql
```

Your result:

```
-rw-r--r-- 1 root root 2.2K ... devopsdb_backup.sql
```

4. Check the beginning of the backup:

```
head devopsdb_backup.sql
```

It confirmed:

```
Database: devopsdb
```

```
Server version: 10.5.29-MariaDB
```

5. Verify that the dummy records are included:

```
grep "INSERT INTO" devopsdb_backup.sql
```

Your output contained:

```
INSERT INTO `employees` VALUES
(1,'Chandra','DevOps',50000.00),
(2,'Rahul','Cloud',55000.00),
(3,'Priya','Database',60000.00),
(4,'Arjun','Linux',48000.00),
(5,'Sneha','AWS',58000.00);
```

Result: `devopsdb` was successfully backed up on the EC2 instance as `devopsdb_backup.sql`, and the backup contains the employee table data. 

```

root@ip-172-31-21-20:/home/ec2-user
+-----+
| Tables_in_devopsdb |
+-----+
| employees           |
+-----+
1 row in set (0.000 sec)

MariaDB [devopsdb]> INSERT INTO employees (name, department, salary) VALUES
  -> ('Chandra', 'DevOps', 50000.00),
  -> ('Rahul', 'Cloud', 55000.00),
  -> ('Priya', 'Database', 60000.00),
  -> ('Arjun', 'Linux', 48000.00),
  -> ('Sneha', 'AWS', 58000.00);
Query OK, 5 rows affected (0.002 sec)
Records: 5 Duplicates: 0 Warnings: 0

MariaDB [devopsdb]> SELECT * FROM employees;
+----+-----+-----+-----+
| id | name  | department | salary |
+----+-----+-----+-----+
| 1  | Chandra | DevOps      | 50000.00 |
| 2  | Rahul  | Cloud       | 55000.00 |
| 3  | Priya  | Database    | 60000.00 |
| 4  | Arjun  | Linux       | 48000.00 |
| 5  | Sneha  | AWS         | 58000.00 |
+----+-----+-----+-----+
5 rows in set (0.000 sec)

MariaDB [devopsdb]> exit;
Bye
[root@ip-172-31-21-20 ec2-user]# mariadb-dump -u root devopsdb > devopsdb_backup.sql
[root@ip-172-31-21-20 ec2-user]# ls -lh devopsdb_backup.sql
-rw-r--r-- 1 root root 2.2K Sep 24 11:46 devopsdb_backup.sql
[root@ip-172-31-21-20 ec2-user]# head devopsdb_backup.sql
/*M19999999~ enable the sandbox mode */
-- MariaDB dump 10.19 Distrib 10.5.29-MariaDB, for Linux (x86_64)
--
-- Host: localhost    Database: devopsdb
--
-- Server version      10.5.29-MariaDB
--
/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
[root@ip-172-31-21-20 ec2-user]# head devopsdb_backup.sql
/*M19999999~ enable the sandbox mode */
-- MariaDB dump 10.19 Distrib 10.5.29-MariaDB, for Linux (x86_64)
--
-- Host: localhost    Database: devopsdb
--
-- Server version      10.5.29-MariaDB
--
/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
[root@ip-172-31-21-20 ec2-user]# grep "INSERT INTO" devopsdb_backup.sql
INSERT INTO `employees` VALUES (1,'Chandra','DevOps',50000.00),(2,'Rahul','Cloud',55000.00),(3,'Priya','Database',60000.00),(4,'Arjun','Linux',48000.00),(5,'Sneha','AWS',58000.00);
[root@ip-172-31-21-20 ec2-user]#

```

4.Launch MariaDB RDS instance.

Task 4 – Launch MariaDB RDS Instance

Objective

Create an Amazon RDS MariaDB database instance and configure connectivity between the existing EC2 instance and RDS.

Steps

1. Go to **AWS Console** → **Amazon RDS** → **Databases** → **Create database**.
2. Select:
 - **Engine:** MariaDB
 - **Creation method:** Full configuration
 - **Template:** Free tier
3. Configure the RDS instance:
 - **DB instance class:** db.t4g.micro
 - **Storage:** 20 GiB
 - **Storage type:** General Purpose SSD
 - **Multi-AZ:** Disabled for this task

4. Configure credentials:
 - **Master username:** `admin`
 - **Credentials:** Self managed
 - Set and confirm the master password.
5. Under **Connectivity**, select:
 - **Connect to an EC2 compute resource**
 - Select the existing `MariaDB-EC2` instance
 - **Network type:** IPv4
 - **VPC:** Same VPC as the EC2 instance
 - **Public access:** No
6. Keep **RDS Proxy** disabled and use the default database port:

3306

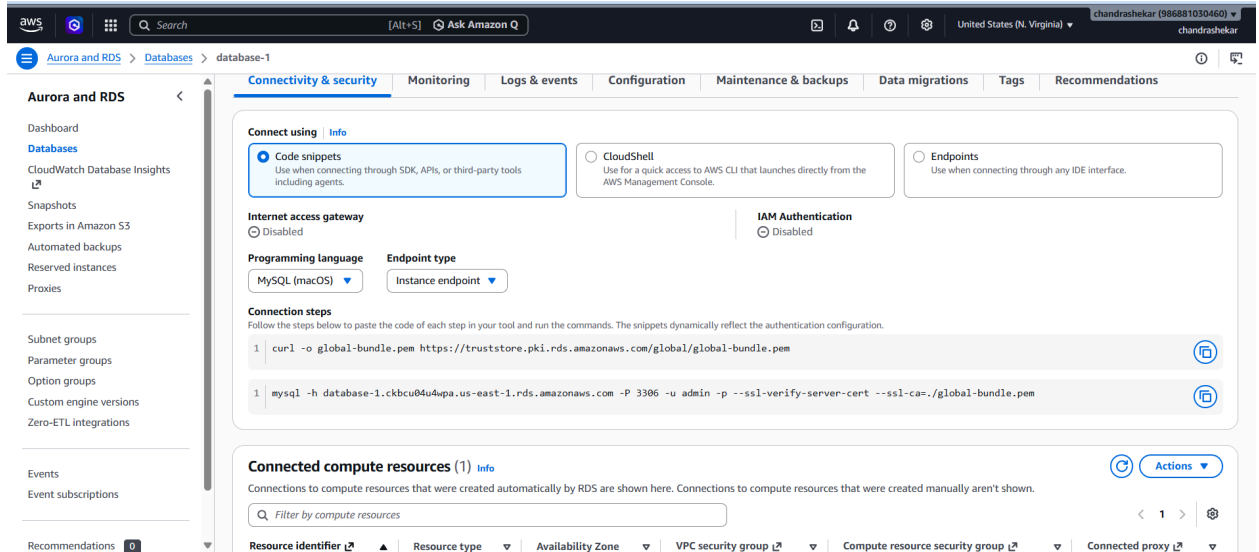
7. Under additional configuration:
 - Encryption: Enabled
 - Automated backups: Enabled
 - Backup retention: `1 day`
 - Enhanced Monitoring: Disabled
 - Deletion protection: Disabled for this lab
8. Click **Create database**.
9. Wait until the RDS status changes:

Creating → Available

10. AWS successfully configured connectivity between the RDS database and the EC2 instance.

Result

MariaDB RDS instance successfully launched and became Available.



5.Migrate database from EC2 to RDS.

Task 5 – Migrate Database from EC2 to RDS

Objective

Migrate the **devopsdb** database and its dummy employee data from MariaDB running on EC2 to the MariaDB RDS instance.

Step 1 – Download the RDS CA Certificate

On the EC2 instance:

```
curl -o global-bundle.pem  
https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
```

Step 2 – Connect from EC2 to MariaDB RDS

We connected using the RDS endpoint:

```
mysql -h database-1.ckbcu04u4wpa.us-east-1.rds.amazonaws.com -P 3306 -u admin -p  
--ssl-verify-server-cert --ssl-ca=./global-bundle.pem
```

Enter the RDS master password when prompted.

Successful connection showed:

```
Server version: 11.8.8-MariaDB-log  
managed by AWS RDS
```

Step 3 – Check Existing Databases

```
SHOW DATABASES;
```

Initially, `devopsdb` was not present on RDS.

Step 4 – Create Database on RDS

```
CREATE DATABASE devopsdb;
```

Verify:

```
SHOW DATABASES;
```

`devopsdb` now appeared in the database list.

Exit RDS:

```
exit;
```

Step 5 – Verify the EC2 Backup

The backup from Task 3 was:

```
devopsdb_backup.sql
```

Verify it:

```
ls -lh devopsdb_backup.sql
```

Our backup size was approximately:

```
2.2K
```

Step 6 – Import EC2 Backup into RDS

Run:

```
mysql -h database-1.ckbcu04u4wpa.us-east-1.rds.amazonaws.com -P 3306 -u admin -p  
--ssl-verify-server-cert --ssl-ca=./global-bundle.pem devopsdb < devopsdb_backup.sql
```

Enter the RDS password.

The command returned without an error, indicating the SQL dump was imported.

Step 7 – Reconnect to RDS

```
mysql -h database-1.ckbcu04u4wpa.us-east-1.rds.amazonaws.com -P 3306 -u admin -p  
--ssl-verify-server-cert --ssl-ca=./global-bundle.pem
```

Step 8 – Verify Migrated Database

Select the database:

```
USE devopsdb;
```

Check the tables:

```
SHOW TABLES;
```

Result:

```
+-----+  
| Tables_in_devopsdb |  
+-----+  
| employees          |  
+-----+
```

Step 9 – Verify Migrated Data

```
SELECT * FROM employees;
```

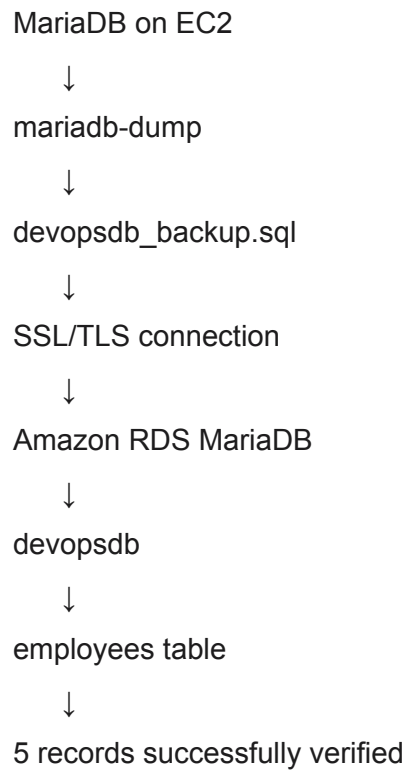
Result:

```
+----+-----+-----+-----+  
| id | name  | department | salary |  
+----+-----+-----+-----+  
| 1  | Chandra | DevOps    | 50000.00 |  
| 2  | Rahul  | Cloud     | 55000.00 |  
| 3  | Priya  | Database  | 60000.00 |  
| 4  | Arjun  | Linux     | 48000.00 |  
| 5  | Sneha  | AWS       | 58000.00 |  
+----+-----+-----+-----+
```

Result

Database migration completed successfully. ✓

The complete migration flow was:



root@ip-172-31-21-20:/home/ec2-user

```
+-----+
| Database |
+-----+
| devopsdb |
| information_schema |
| innodb |
| mysql |
| performance_schema |
| sys |
+-----+
6 rows in set (0.001 sec)
```

MariaDB [(none)]> exit;

Bye

[root@ip-172-31-21-20 ec2-user]# devopsdb_backup.sql

bash: devopsdb_backup.sql: command not found

[root@ip-172-31-21-20 ec2-user]# ls -lh devopsdb_backup.sql

-rw-r--r--. 1 root root 2.2K Sep 24 11:46 devopsdb_backup.sql

[root@ip-172-31-21-20 ec2-user]# mysql -h database-1.ckbcu04u4wpa.us-east-1.rds.amazonaws.com -u root -p

Enter password:
[root@ip-172-31-21-20 ec2-user]# mysql -h database-1.ckbcu04u4wpa.us-east-1.rds.amazonaws.com -u root -p

Enter password:

Welcome to the MariaDB monitor. Commands end with ; or \g.

Your MariaDB connection id is 71

Server version: 11.8.8-MariaDB-log managed by <https://aws.amazon.com/rds/>

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> USE devopsdb;

Reading table information for completion of table and column names

You can turn off this feature to get a quicker startup with -A

Database changed

MariaDB [devopsdb]>

MariaDB [devopsdb]> SHOW TABLES;

```
+-----+
| Tables_in_devopsdb |
+-----+
| employees |
+-----+
1 row in set (0.000 sec)
```

MariaDB [devopsdb]>

MariaDB [devopsdb]> SELECT * FROM employees;

```
+----+-----+-----+-----+
| id | name  | department | salary |
+----+-----+-----+-----+
| 1  | Chandra | DevOps      | 50000.00 |
| 2  | Rahul  | Cloud       | 55000.00 |
| 3  | Priya  | Database    | 60000.00 |
| 4  | Arjun  | Linux       | 48000.00 |
| 5  | Sneha  | AWS         | 58000.00 |
+----+-----+-----+-----+
5 rows in set (0.001 sec)
```

MariaDB [devopsdb]> |

6.Install MySQL DB on EC2.

Task 6 – Install MySQL Database on EC2

Objective

Launch a fresh EC2 instance, install **MySQL Community Server**, start the MySQL service, configure the root password, and create a MySQL database.

Step 1 – Launch EC2 Instance

Go to:

AWS Console → EC2 → Instances → Launch instances

Configure:

Name : MySQL-EC2
AMI : Amazon Linux 2023
Instance Type : t3.micro
Key Pair : my_server
Public IP : Enabled
Storage : 8 GiB gp3

Create/use a security group allowing **SSH port 22** from your IP.

Launch the instance and wait until the status checks pass.

Step 2 – Connect to EC2

From Git Bash:

```
chmod 400 "my_server.pem"
```

Connect:

```
ssh -i "my_server.pem" ec2-user@<EC2-PUBLIC-DNS>
```

Switch to root:

```
sudo su
```

Step 3 – Update the Server

```
dnf update -y
```

Step 4 – Add MySQL Community Repository

```
dnf install -y https://dev.mysql.com/get/mysql80-community-release-el9-1.noarch.rpm
```

During our installation, the MySQL package initially failed with a **GPG check error**.

We fixed it by importing the current MySQL signing key:

```
dnf clean all
```

```
rpm --import https://repo.mysql.com/RPM-GPG-KEY-mysql-2025
```

```
dnf clean all
```

```
dnf makecache
```

Step 5 – Install MySQL Server

```
dnf install -y mysql-community-server
```

Installation completed successfully with MySQL:

```
mysql-community-server-8.0.46
```

Step 6 – Start and Enable MySQL

```
systemctl start mysqld
```

```
systemctl enable mysqld
```

Check status:

```
systemctl status mysqld
```

Result:

```
Active: active (running)
```

```
Status: "Server is operational"
```

This confirms that the MySQL server is running successfully.

Step 7 – Find Temporary Root Password

MySQL generates a temporary root password during the initial startup.

Check it using:

```
grep 'temporary password' /var/log/mysqld.log
```

For documentation, **do not write the actual password**. Mention only:

Temporary root password generated successfully.

Step 8 – Configure Root Password

Login:

```
mysql -u root -p
```

If the temporary password works normally, change it immediately with:

```
ALTER USER 'root'@'localhost' IDENTIFIED BY '<NEW-STRONG-PASSWORD>';
```

In our lab, the temporary password was rejected, so we reset it using MySQL's recovery mode.

Stop MySQL:

```
systemctl stop mysqld
```

Start temporarily without privilege checking/network access:

```
systemctl set-environment MYSQLD_OPTS="--skip-grant-tables --skip-networking"
systemctl start mysqld
```

Login without a password:

```
mysql -u root
```

Then:

```
FLUSH PRIVILEGES;
ALTER USER 'root'@'localhost' IDENTIFIED BY '<NEW-STRONG-PASSWORD>';
exit;
```

In documentation, use **<NEW-STRONG-PASSWORD>** rather than putting your real password.

Step 9 – Return MySQL to Normal Mode

Stop MySQL and remove the temporary recovery option:

```
systemctl stop mysqld
systemctl unset-environment MYSQLD_OPTS
systemctl start mysqld
```

Login normally:

```
mysql -u root -p
```

Successful login displayed:

```
Welcome to the MySQL monitor.
Server version: 8.0.46 MySQL Community Server - GPL
mysql>
```

Step 10 – Create MySQL Database

Inside MySQL:

```
CREATE DATABASE mysqlpdb;
```

Verify:

```
SHOW DATABASES;
```

Confirm that:

```
mysqlpdb
```

appears in the database list.

Result

Task 6 completed successfully. ✓

The final setup is:

```
Amazon Linux 2023 EC2
↓
MySQL Community Server 8.0.46
↓
mysqld service
↓
Root authentication configured
```

mysqlldb created

Successfully initiated stopping of i-0af0ed2e2327db872

Instances (1/1) Info

Last updated less than a minute ago

Connect Instance state Actions Launch instances

Saved filter sets: Running Filter instance by attribute or tag (case-sensitive)

Instance state = running Clear filters

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv
MySQL-EC2	i-0221c66c131f9669e	Running	t3.micro	3/3 checks passed	us-east-1a	ec2-98-84-48-118.com...	98.84.48.1

i-0221c66c131f9669e (MySQL-EC2)

Details Status and alarms Monitoring Security Networking Storage Tags

Instance summary Info

Instance ID i-0221c66c131f9669e	Public IPv4 address 98.84.48.118 open address	Private IPv4 addresses 172.31.7.72
IPv6 address -	Instance state Running	Public DNS ec2-98-84-48-118.compute-1.amazonaws.com open address
Hostname type IP name: ip-172-31-7-72.ec2.internal	Private IP DNS name (IPv4 only) ip-172-31-7-72.ec2.internal	Elastic IP addresses -
Answer private resource DNS name IPv4 (A)	Instance type t3.micro	AWS Compute Optimizer finding
Auto-assigned IP address	VPC ID	

```

root@ip-172-31-7-72:/home/ec2-user
systemctl start mysqld
[root@ip-172-31-7-72 ec2-user]# mysql -u root
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 7
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.02 sec)

mysql> ALTER USER 'root'@'localhost' IDENTIFIED BY 'DevOps@2026#Mysql';
Query OK, 0 rows affected (0.01 sec)

mysql> exit;
Bye
[root@ip-172-31-7-72 ec2-user]# systemctl stop mysqld
systemctl unset-environment MYSQLD_OPTS
systemctl start mysqld
[root@ip-172-31-7-72 ec2-user]# mysql -u root -p
Enter password:
ERROR 1045 (28000): Access denied for user 'root'@'localhost' (using password: YES)
[root@ip-172-31-7-72 ec2-user]# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE mysqldb;
Query OK, 1 row affected (0.01 sec)

mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| mysqldb |
| performance_schema |
| sys |
+-----+
5 rows in set (0.00 sec)

mysql>

```

7.Launch MySQL RDS image.

Task 7 – Launch MySQL Database Using Amazon RDS

Objective

Create a **MySQL database using Amazon RDS** and configure it to communicate with the previously created **MySQL-EC2** instance.

Step 1 – Open Amazon RDS

Login to the AWS Management Console.

Navigate to:

AWS Console → Aurora and RDS → Databases → Create database

Step 2 – Select Database Creation Method

Select:

Create database → Full configuration

Full configuration allows us to configure the database engine, instance class, storage, connectivity, backup, monitoring, and other options.

Step 3 – Select MySQL Engine

Under **Engine options**, select:

Engine type: MySQL

Use the available/default MySQL engine version.

Step 4 – Select Template

For this practice environment, select:

Template: Free tier

This provides a small database instance suitable for learning and testing.

Step 5 – Configure Database Settings

Configure the database details.

DB instance identifier : database-2
Master username : admin
Credentials management : Self managed

Enter a strong master password and confirm it.

Do not include the actual database password in your documentation or screenshots.

Step 6 – Configure DB Instance

Under **Instance configuration**, select:

DB instance class : Burstable classes
Instance type : db.t4g.micro

This is the instance class used in our task.

Step 7 – Configure Storage

Configure:

Storage type : General Purpose SSD
Allocated storage : 20 GiB

For this practice task, the minimum/free-tier configuration is sufficient.

Step 8 – Availability and Durability

For the initial RDS creation, we did not configure a standby Multi-AZ instance.

Multi-AZ deployment:
Do not create a standby instance

Multi-AZ can be practiced separately as another task.

Step 9 – Configure EC2 Connectivity

Under **Connectivity** → **Compute resource**, select:

Connect to an EC2 compute resource

Select the EC2 instance created for MySQL:

MySQL-EC2

AWS automatically configures connectivity between the selected EC2 instance and the RDS database using VPC security groups.

Step 10 – Network Configuration

The configuration used was:

Network type : IPv4
VPC : Default VPC
DB subnet group : Automatic setup
Public access : No

Keeping **Public access = No** means the RDS database is not directly exposed to the internet. The EC2 instance inside the VPC is used to access it.

Step 11 – Security Group

Because we selected **Connect to an EC2 compute resource**, AWS configured the required VPC security-group relationship between EC2 and RDS.

MySQL uses:

TCP Port: 3306

Step 12 – RDS Proxy and Certificate

Keep:

Create an RDS Proxy : Unchecked
Certificate authority: Default

RDS Proxy was not required for this basic task.

Step 13 – Monitoring

Use:

Database Insights : Standard
Enhanced Monitoring : Disabled
CloudWatch Log Exports : Not selected

These settings are sufficient for this practice environment.

Step 14 – Additional Configuration

For the basic lab configuration:

Encryption : Enabled/default
Automated backups : Enabled
Backup retention : 1 day
Backup window : No preference
Auto minor upgrade : Default/Enabled
Maintenance window : No preference
Deletion protection : Disabled

Step 15 – Create the RDS Database

Review the configuration and click:

Create database

AWS starts provisioning the MySQL RDS instance.

Wait until the database status changes:

Creating
↓
Available

Step 16 – Verify RDS Instance

Go to:

RDS → Databases

Our final result showed:

DB Identifier : database-2
Status : Available
Engine : MySQL Community
Role : Instance
DB Class : db.t4g.micro
AZ : us-east-1a

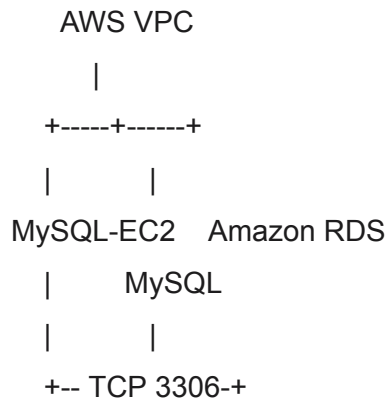
AWS also displayed:

Successfully set up a connection between

database-2 and EC2 instance i-0221c66c131f9669e

This confirms that the RDS database was successfully created and its connection with the **MySQL-EC2** instance was configured.

Final Architecture



Task 7 completed successfully. ✓

Aurora and RDS > Databases

Successfully set up a connection between database-2 and EC2 instance i-0221c66c131f9669e

Databases (2)

Filter by databases

DB identifier	Status	Role	Engine	Upgrade rollout order	Region ...	Size	Recommendation
database-1	Available	Instance	MariaDB	SECOND	us-east-1c	db.t4g.micro	2 Informational
database-2	Available	Instance	MySQL Co...	SECOND	us-east-1a	db.t4g.micro	

```

root@ip-172-31-7-72:/home/ec2-user
systemctl start mysqld
[root@ip-172-31-7-72 ec2-user]# mysql -u root
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 7
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.02 sec)

mysql> ALTER USER 'root'@'localhost' IDENTIFIED BY 'DevOps@2026#Mysql';
Query OK, 0 rows affected (0.01 sec)

mysql> exit;
Bye
[root@ip-172-31-7-72 ec2-user]# systemctl stop mysqld
systemctl unset-environment MYSQLD_OPTS
systemctl start mysqld
[root@ip-172-31-7-72 ec2-user]# mysql -u root -p
Enter password:
ERROR 1045 (28000): Access denied for user 'root'@'localhost' (using password: YES)
[root@ip-172-31-7-72 ec2-user]# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE mysqlpdb;
Query OK, 1 row affected (0.01 sec)

mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| mysqlpdb |
| performance_schema |
| sys |
+-----+
5 rows in set (0.00 sec)

mysql> |

```

8.Configure Multi-AZ.

Task 8 – Configure Multi-AZ Deployment in Amazon RDS

Objective

Configure an Amazon RDS database for **Multi-AZ deployment** to improve database availability and provide automatic failover.

Step 1 – Open Amazon RDS

Login to AWS Console and navigate to:

AWS Console
→ Aurora and RDS
→ Databases

Step 2 – Select the MySQL RDS Database

Select the MySQL RDS instance created in Task 7:

DB Identifier : database-2
Engine : MySQL
Status : Available

Step 3 – Modify the Database

Select:

database-2
→ Modify

This opens the configuration page for the existing RDS instance.

Step 4 – Find Availability & Durability

Scroll down to:

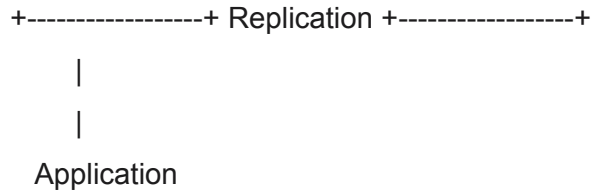
Availability & durability

Look for the **Multi-AZ deployment** setting.

Normally, for a supported configuration, we can select an option to create a **standby DB instance in another Availability Zone**.

Conceptually:

Availability Zone 1		Availability Zone 2
+-----+		+-----+
Primary RDS DB	----->	Standby RDS DB
MySQL	Synchronous	MySQL



Step 5 – Check Multi-AZ Availability

In our current lab configuration, the Multi-AZ option was **disabled/unavailable**.

The RDS instance was therefore kept as:

Deployment : Single-AZ

Multi-AZ : Not enabled

We did **not upgrade the AWS plan or select a larger paid configuration** just to perform this lab.

Step 6 – What We Would Do If Multi-AZ Were Available

For a supported configuration, the procedure would be:

RDS

→ Databases

→ Select database

→ Modify

→ Availability & durability

→ Enable Multi-AZ / Create standby

→ Continue

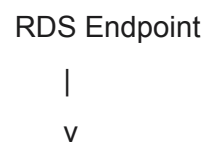
→ Apply immediately (for lab)

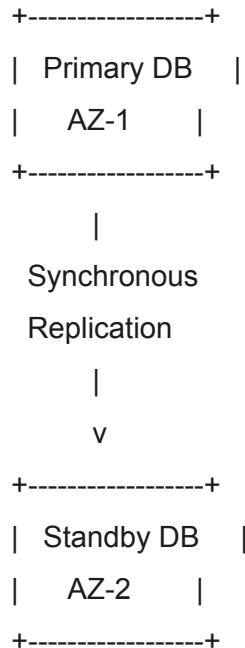
→ Modify DB instance

AWS would then create a standby database in another Availability Zone.

Step 7 – Understand Multi-AZ Failover

With Multi-AZ:





If the primary database has a qualifying failure, RDS can automatically fail over to the standby.

The application continues using the **RDS endpoint**, rather than connecting directly to a particular DB instance IP.

Result

Task 8 – Multi-AZ configuration reviewed.

For our practical AWS account/configuration:

MySQL RDS : database-2

Current setup : Single-AZ

Multi-AZ : Not enabled

Reason : Option unavailable for current Free plan/configuration

Learning outcome: Multi-AZ is mainly used for **high availability and automatic failover**, not as a read-scaling solution.

Configure Multi-AZ Deployment: Multi-AZ was reviewed for Amazon RDS. In the current AWS Free plan, the Multi-AZ standby option was unavailable/disabled for the selected configuration. Therefore, the existing MySQL RDS instance was kept as a Single-AZ deployment. In a production environment, Multi-AZ can be enabled to maintain a synchronous standby DB instance in another Availability Zone and provide automatic failover for higher availability.

9. Take backup of DB and restore the DB.

Task 9 – Take Backup of RDS Database and Restore the Database

Objective

Take a backup of the existing **MySQL Amazon RDS database** using a manual snapshot and restore that snapshot as a new RDS DB instance.

Step 1 – Open Amazon RDS

Login to AWS Console and navigate to:

AWS Console

→ Aurora and RDS

→ Databases

Our source database was:

DB Identifier : database-2

Engine : MySQL

Status : Available

Step 2 – Select the Database

Select:

database-2

Then choose:

Actions

→ Take snapshot

Step 3 – Create Manual Snapshot

Enter the snapshot name:

mysql-db-backup

Click:

Take snapshot

Step 4 – Verify the Backup

Navigate to:

RDS

→ Snapshots

→ Manual

The manual snapshot was successfully created:

Snapshot name : mysql-db-backup

Source DB : database-2

Engine : MySQL

Wait until the snapshot is ready before restoring it.

Step 5 – Start Database Restore

Select:

mysql-db-backup

Then:

Actions

→ Restore snapshot

Amazon RDS opens the **Restore snapshot** configuration page.

Step 6 – Configure Restored Database

We used:

DB Engine : MySQL Community

Deployment : Single-AZ

DB Instance Identifier: database-2-restored

We kept **RDS Extended Support** disabled.

Step 7 – Configure Instance and Storage

For the restored database:

DB Instance Class : Burstable
Instance Type : db.t3.micro
Storage Type : General Purpose SSD (gp2)

The storage configuration was inherited/configured from the snapshot.

Step 8 – Configure Network

We kept the database in the existing VPC configuration:

Network type : IPv4
VPC : Default VPC
DB Subnet Group : Existing RDS subnet group
Public Access : No

The database therefore remains accessible through appropriate resources/security groups inside the VPC.

Step 9 – Database Authentication and Encryption

We kept:

Authentication : Password authentication
Encryption : Enabled
KMS Key : Default AWS RDS key

Other settings were left at their appropriate/default values for this lab.

Step 10 – Restore DB Instance

Click:

Restore DB instance

AWS started creating a new database from the snapshot.

The console displayed:

Snapshot is being restored

Restoring mysql-db-backup

to database-2-restored

Step 11 – Verify Restore

Navigate to:

RDS

→ Databases

We could see:

database-2 → Available

database-2-restored → Creating

After restoration finishes, the expected status is:

database-2-restored → Available

This confirms that the RDS database has been successfully restored from the backup.

Task 9 Flow

Original MySQL RDS

database-2

|

| Take Snapshot

↓

mysql-db-backup

|

| Restore Snapshot

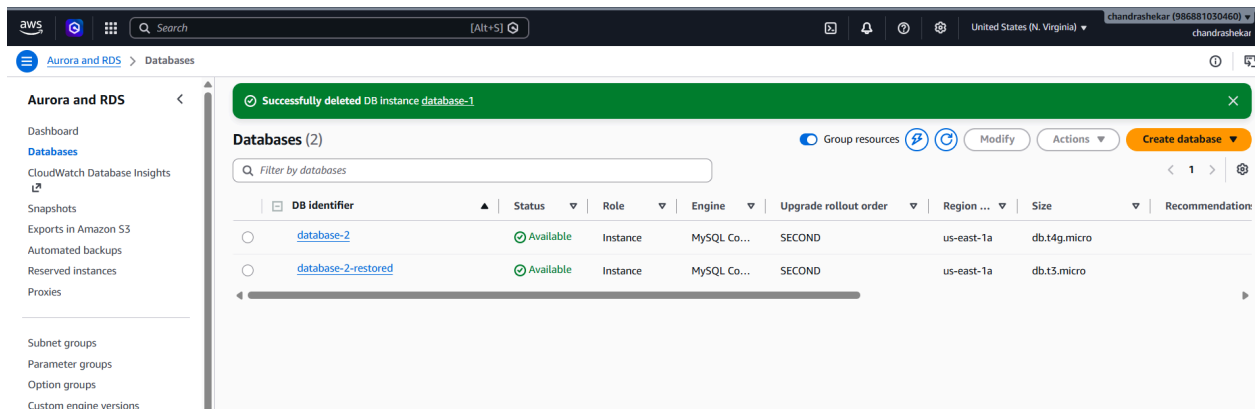
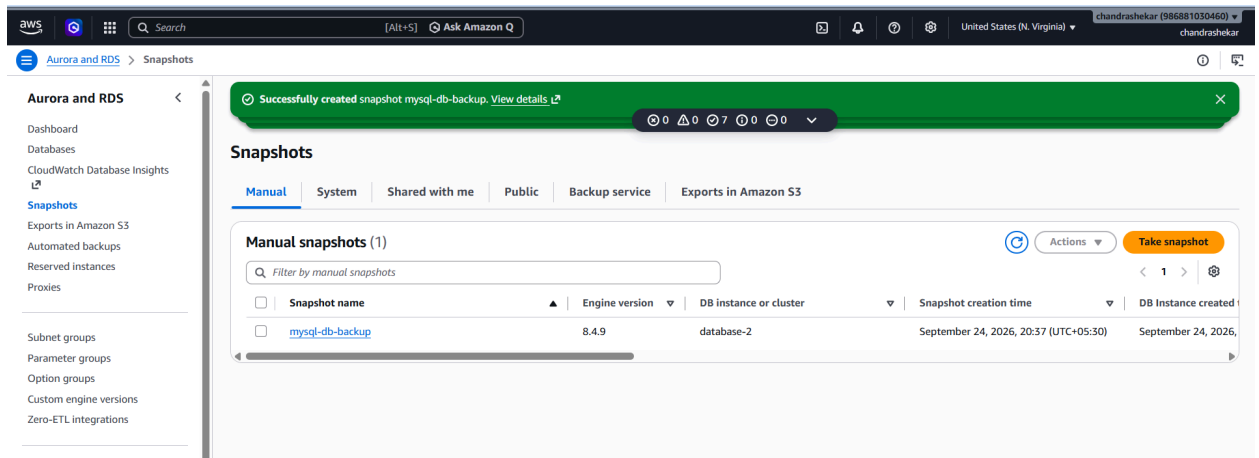
↓

database-2-restored

Result

Task 9 completed: A manual snapshot named `mysql-db-backup` was taken from the MySQL RDS instance `database-2`, and restoration was initiated as a new RDS instance named `database-2-restored`.

Important: For final proof, take a screenshot once `database-2-restored` changes from **Creating** → **Available**.



10.Create read replica.

Task 10: Create Read Replica

1. Opened Amazon RDS → Databases.
2. Selected the source MySQL database **database-2**.
3. Selected **Actions** → **Create read replica**.
4. Entered replica identifier as **database-2-read-replica**.
5. Selected **db.t4g.micro**.
6. Kept Enhanced Monitoring disabled.
7. Clicked **Create read replica**.
8. AWS created a read replica from the primary database.

9. Verified **database-2** has **Primary** role and **database-2-read-replica** has **Replica** role.

Purpose: A Read Replica maintains a replicated copy of the source database and can be used for **read workloads**, reducing read load on the primary database.

The screenshot displays the AWS Management Console interface for the 'Databases' section. A prominent blue banner at the top states: 'Creating replica database-2-read-replica in US East (N. Virginia). Your database might take a few minutes to launch.' Below this, the 'Databases (2)' section is visible, featuring a search bar and a table of database instances. The table has columns for DB identifier, Status, Role, Engine, Upgrade rollout order, Region, Size, and Recommendation. Two databases are listed: 'database-2' with a status of 'Available', role of 'Primary', engine of 'MySQL Co...', and 'database-2-read-replica' with a status of 'Config...', role of 'Replica', engine of 'MySQL Co...', and both are in the 'us-east-1a' region with a size of 'db.t4g.micro'. The left sidebar contains navigation links for 'Aurora and RDS', 'Databases', 'CloudWatch Database Insights', 'Snapshots', 'Exports in Amazon S3', 'Automated backups', 'Reserved instances', 'Proxies', 'Subnet groups', 'Parameter groups', 'Option groups', and 'Custom engine versions'.

DB identifier	Status	Role	Engine	Upgrade rollout order	Region	Size	Recommendation
database-2	Available	Primary	MySQL Co...	SECOND	us-east-1a	db.t4g.micro	
database-2-read-replica	Config...	Replica	MySQL Co...	SECOND	us-east-1c	db.t4g.micro	